MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Your first steps in programming and numeric computing with MATLAB

Under the supervision of **DR. Lobna Abouelmagd**

Prepared by **ENG. Moamen Yasser**

BEFORE WE START

What is MATLAB?

MATLAB is a programming and numeric computing environment used by engineers and scientists to analyze data, develop algorithms, and create models and plots.

WHERE THE NAME COMES FROM

MATrix LABoratory

Cleve Moler created it in the late 1970s so students could use matrix software without writing Fortran. MathWorks was founded in 1984.

BEFORE WE START

Why use MATLAB?

- Matrix based: arrays are the basic data type, so working on whole data sets takes very little code.
- Interactive: type a command and see the result immediately.
- Built-in functions for math, data analysis and plotting.
- Toolboxes add specialized tools for areas such as signal processing, control and machine learning.

MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Part 1: Enter Commands

Commands · Part 1 of 10

TASK 1

Run a command

Type a command in the **Command Window** after the **>>** prompt, then press **Enter**. MATLAB runs it and shows the result.

```
>> 3*5
ans =
    15
```

multiply 3 and 5 with the command 3*5

TASK 2

Store a result in a variable

Unless you name an output variable, MATLAB stores the result in a variable called **ans**. To keep a result under your own name, write the name, then **=**, then the expression.

```
>> 7 + 3
ans =
    10
>> m = 3*5
```

assign the result of 3*5 to a variable named m

TASK 3

The assignment operator

The equal sign **=** is not a math equation. MATLAB evaluates the **right side** first, then assigns the result to the variable on the **left**. So m = m + 1 reads the old m, adds 1, and stores the new value back in m.

```
>> x = 3 + 4
x =
     7
```

enter m = m + 1 and watch what happens to m

TASK 4

The Workspace panel

The **Workspace** panel, on the right, lists every variable you have created, with its value, size and class.

Name	Value	Size	Class
ans	15	1x1	double
m	16	1x1	double

create a variable y with the value of m/2

TASK 5

Suppress output with a semicolon

Without a semicolon, MATLAB displays the result. With a **;** at the end it still runs the command, but hides the output. The variable still appears in the Workspace.

```
>> x = 5 + 1
x =
     6
>> x = 5 + 1;
```

enter k = 8 - 2; then find k in the Workspace panel

TASK 6

Recall previous commands

Press the **Up arrow** key to bring back earlier commands, then edit and run them. The Command Window must be the active window for this to work.

```
>> m = 3*5
     ↑ edit
>> m = 3*k
```

press Up to recall m = 3*5, change it to m = 3*k, then press Enter

TASK 7 + FURTHER PRACTICE

Variables keep their values

Type just a variable name to see its current value. y was calculated from the old m, so it is **unchanged** after m changed: MATLAB does not rerun earlier commands. To update y, run **y = m/2** again.

```
>> y
y =
     8
>> y = m/2
y =
     9
```

type y and press Enter. Then recall y = m/2 with the Up arrow and run it without a semicolon.

MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Part 2: Name Variables

Commands · Part 2 of 10

TASK 1

Variable naming rules

A name must **start with a letter** and contain only letters (A-Z, a-z), numbers and underscores (_). Variable names are also **case sensitive**, so a and A are two different variables.

```
>> A = -2
A =
    -2
```

create a variable named A with the value -2

TASK 2

Choose meaningful names

Both **a** and **A** now exist in the Workspace. Single letters work, but a meaningful name such as **meanAa** makes your code easier to read.

Note: a = 8

```
>> meanAa = (a + A)/2
```

Lab task: use A and a to calculate (a + A)/2 and store the result in a variable named meanAa

FURTHER PRACTICE

Invalid variable names

With an invalid name, MATLAB shows an **error message** and a **suggested correction**. Use the correction, modify it, or press **Esc** to delete the suggestion.

```
>> 3sq = 9
Error + suggested fix
```

try creating 3sq = 9 and read the error message and the suggested correction

MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Part 3: Save and Load Variables

Commands · Part 3 of 10

TASK 1

Save variables to a MAT-file

Use the **save** command to store the variables in your workspace in a **MAT-file**, a file format specific to MATLAB. To save everything to filename.mat, type save followed by the file name.

```
>> save filename
```

save all the variables in the workspace to a file named datafile.mat

TASK 2

Empty the workspace

When you switch to a new problem, you may want to tidy up your workspace. The **clear** command removes all the variables from it.

```
>> clear
```

empty the workspace using the clear command

TASK 3

Load variables from a MAT-file

Look at the Workspace panel: **clear** removed all the variables. To bring variables back from a MAT-file, use the **load** command.

```
>> load filename
```

load the variables from the file datafile.mat

TASK 4

Display a loaded variable

The variable **data** is now listed in the Workspace panel. To see the contents of any variable, enter its name in the Command Window.

```
>> myvar
```

display the contents of the variable data

TASK 5

Clear the Command Window

The **clear** command cleans up the workspace. The **clc** command cleans up the Command Window.

```
>> clear
>> clc
```

clear the Command Window using the clc command

FURTHER PRACTICE

Save and load only some variables

Closing MATLAB clears the workspace, so save your variables in MAT-files first. To load or save only **some** variables, add their names after the file name.

```
>> load myData k
>> save justk k
```

load just the variable k from myData.mat, then save k to a new MAT-file named justk.mat

MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Part 4: Use Built-in Functions and Constants

Commands · Part 4 of 10

TASK 1

Built-in constants

MATLAB contains built-in constants, such as **pi** for π. The Command Window shows only four decimal places, but MATLAB stores the constant with more.

```
>> a = pi
a =
    3.1416
```

Lab task: create a variable named x with a value of π/2

TASK 2

Built-in functions

MATLAB has a wide variety of built-in functions, such as **abs** (absolute value) and **eig** (eigenvalues). Pass inputs to a function inside parentheses, like function notation in math.

```
>> a = sin(-5)
a =
    0.9589
```

Lab task: calculate the sine of x with the sin function and assign the result to a variable named y

TASK 3

Calculate a square root

The **sqrt** function works like sin: put the input in parentheses. Check the Workspace to see how the result is stored.

```
>> z = sqrt(-9)
```

Lab task: calculate the square root of -9 with the sqrt function and assign the result to a variable named z

FURTHER PRACTICE

Imaginary numbers and display format

The square root of -9 contains the imaginary number **i**, which is also a built-in constant. The Command Window shows only four decimal places; the **format** function controls how many are displayed.

```
>> format long
>> x
>> format short
>> x
```

Lab task: display more decimal places of x with format long, then switch back with format short

MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Part 5: Scripts

Programming · Part 5 of 10

TASK 1

What is a script?

A **script** is a file of MATLAB commands saved with the **.m** extension. When you run it, MATLAB executes the commands in order, so you can reuse them without retyping.

```
% myscript.m
x = 5;
y = x*2
```

a script holds the same commands you would type in the Command Window

TASK 2

Create and save a script

On the **Home** tab click **New Script**, or type **edit** followed by a name. Write your commands in the Editor, then click **Save**. The name follows the variable naming rules.

```
>> edit myscript
```

create a new script named myscript.m and save it in the Current Folder

TASK 3

Run a script

Click **Run** in the Editor, or type the script name, without .m, in the Command Window. The variables the script creates appear in the **Workspace**.

```
>> myscript
y =
    10
```

run myscript from the Command Window and find x and y in the Workspace

TASK 4

Add comments

A line that starts with **%** is a comment. MATLAB ignores it, so use comments to explain what your code does and why.

```
% convert minutes to seconds
mins = 4;
secs = mins*60
```

add a comment above each command in your script describing what it does

FURTHER PRACTICE

Divide a script into sections

Start a line with **%%** to begin a new section. Use **Run Section** in the Editor to run only the section where the cursor is.

```
%% Inputs
a = 4;
%% Calculation
b = a + 1
```

split your script into two sections with %%, then run only the second one

MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Part 6: Arithmetic Operations

Programming · Part 6 of 10

TASK 1

Basic arithmetic operators

MATLAB uses **+** to add, **-** to subtract, ***** to multiply, **/** to divide and **^** for powers. Each one works on numbers and on variables.

```
>> a = 10 + 4
>> b = 10 - 4
>> c = 10*4
>> d = 10/4
>> e = 10^2
```

calculate a = 10 + 4 and c = 10*4, then check both in the Workspace

TASK 2

Order of operations

MATLAB follows the standard order: **parentheses** first, then **powers**, then **multiplication and division**, then **addition and subtraction**.

```
>> 2 + 3*4
ans =
    14
>> (2 + 3)*4
ans =
    20
```

calculate 2 + 3*4, then (2 + 3)*4, and compare the two results

TASK 3

Powers and roots

Use ^ for powers and **sqrt** for square roots. For other roots, **nthroot(x,n)** returns the n-th root of x.

```
>> x = 2^3
x =
     8
>> y = nthroot(27,3)
```

calculate 2^3, then the cube root of 27 with the nthroot function

TASK 4

Element-wise operations

Arrays are MATLAB's basic data type. To apply an operation to every element, put a dot before the operator: **.***, **./** and **.^**.

```
>> a = [1 2 3];
>> a.^2
ans =
     1     4     9
```

create v = [2 4 6], then calculate v.*v and v./2

FURTHER PRACTICE

Remainders and rounding

mod(10,3) returns the remainder 1. **round** gives the nearest integer, **floor** rounds down, **ceil** rounds up and **fix** rounds toward zero: round(2.6) is 3, floor(2.6) is 2, ceil(2.1) is 3, fix(-2.6) is -2.

```
>> mod(10,3)
>> round(2.6)
>> floor(2.6)
>> ceil(2.1)
>> fix(-2.6)
```

find the remainder of 17 divided by 5, then compare floor(-2.5) with fix(-2.5)

MATLAB ONRAMP - SECTION ONE

MATLAB Fundamentals

Part 7: Conditions

Programming · Part 7 of 10

TASK 1

Relational operators

Comparisons return a logical value: **1** for true and **0** for false. Use **==**, **~=**, **<**, **>**, **<=** and **>=**. A single = assigns, while == compares.

```
>> x = 7;
>> x > 5
ans =
  logical
   1
```

test whether x equals 7 with x == 7 and check that the result is logical 1

TASK 2

Logical operators

&& is true when both conditions are true. **||** is true when at least one is true. **~** reverses a value: true becomes false.

```
>> x = 7;
>> x > 5 && x < 10
ans =
  logical
   1
```

write a test that is true when x is below 5 or above 10

TASK 3

The if statement

An **if** block runs its commands only when the condition is true. Every if needs a matching **end**. The disp function displays text.

```
if x > 5
    disp('x is large')
end
```

set x = 8 and write an if statement that displays x is large when x > 5

TASK 4

if-else

Add **else** to run other commands when the condition is false. Exactly one of the two blocks runs.

```
if x > 5
    disp('large')
else
    disp('small')
end
```

run it with x = 3, then with x = 8, and see which message appears

TASK 5

if-elseif-else

MATLAB checks the conditions from the top and runs the first block that is true, then skips the rest. **else** catches everything left over.

```
if x > 10
  disp('high')
elseif x > 5
  disp('medium')
else
  disp('low')
end
```

test it with x = 12, then x = 7, then x = 2 and compare the three messages

TASK 6

The switch statement

switch compares one value against several **case** values. Use it instead of a long if-elseif chain. **otherwise** runs when no case matches.

```
switch n
  case 1
    disp('one')
  otherwise
    disp('other')
end
```

set n = 1, run the switch, then change n to 5 and run it again

FURTHER PRACTICE

switch with text

A switch also works with text. To match one of several values, put them in braces in a single case. Text is matched exactly, so case matters.

```
switch color
  case {'red','pink'}
    disp('warm')
  otherwise
    disp('other')
end
```

set color = 'pink' and check that the message is warm

MATLAB ONRAMP - SECTION ONE

Thank You

Any questions?

Under the supervision of **DR. Lobna Abouelmagd**

Prepared by **ENG. Moamen Yasser**